Study on Network Importance for ML End Application Robustness

Sergei Chuprov*, Leon Reznik*, and Garegin Grigoryan†
*Rochester Institute of Technology, Rochester, NY, USA,
Email: sc1723@rit.edu, leon.reznik@rit.edu
†Alfred University, Alfred, NY, USA,
Email: grigoryan@alfred.edu

***Abstract*—In this paper, we investigate the robustness of the ML end application performance to network Quality of Service (QoS) degradation, and the ways to improve it. We introduce a novel system approach to define the Machine Learning (ML) with Integrated Networks (MLINs) and describe how ML end performance can be employed to adjust network hyperparameters in order to prevent system Data Quality decrease. We investigate the interrelations between the network QoS degradation during the data transmission and ML image classification performance. We demonstrate how the studied interrelationships can be employed to produce recommendations on network adjustment in order to improve MLIN robustness. In particular, we propose an example of MLIN feedback system design that employs ML end performance as the major indicator to produce recommendations on network hyperparameters adjustment aimed at improving MLIN robustness.**

***Index Terms*—network QoS, data quality, machine learning robustness**

I. Introduction

The evolution in the information and communication technologies that has been happening alongside the progress in machine learning (ML), led to combining these two domains in the development of ML with Integrated Networks (MLINs) systems, where computer networks are used to transfer data from sensors to cloud-based remote ML applications. Over this communication, data loss or alteration may occur due to various reasons, for example, malicious network attacks or network congestion that may impact ML end performance and robustness. The integration of data processing, networks, and ML domains makes them inevitably interdependent on each other. So far, all these components have been mostly studied and designed separately, and the interconnections and interrelations between them remain under-researched. In our work, we propose a novel system approach, in which we make MLIN design decisions by considering the influence of its components on network Quality of Service (QoS) and Data Quality (DQ).

Multiple methods that separately address and improve the effectiveness of ML systems have been developed and employed [5], [9], [28]. Some of them are aimed at improving the quality of data inputs (e.g., data cleaning, filtering, or wrangling), while others focus on the ML model (e.g. transfer or adversarial learning, model adjustment) [2]. In our study, we research MLIN from the system integration perspective and propose methods that tie MLIN components together into an interrelated structure with the goal to improve its robustness against potential DQ degradation due to network disruptions. The relationship between network QoS and user quality of experience has been vigorously studied [12], [17], [18]. Some research addresses the effect of network QoS degradation on ML end application performance [6], [27], however, this effect may highly depend on a particular end application and still requires additional research.

In this paper, we propose an innovative MLIN feedback mechanism that allows to dynamically adjust MLIN's network hyperparameters, such as network communication protocol, to improve MLIN robustness by ensuring ML end application performance at the level specified by the user and application requirements. We consider the wireless transmission of media files over Wireless Multimedia Sensor Networks (WMSNs) that are employed to deliver multimedia data, including images, videos, and audio, between various sensor and IoT devices [23]. There is a variety of network protocols used for media file transmission in WMSNs. Some of them utilize UDP at the transport level (e.g., CoAP, MQTT-SN), while others are based on TCP (e.g., MQTT, SMQTT). As IoT devices are usually limited in computational and energy resources, the employment of UDP allows longer device uptime and reduces resource consumption. While not reliable, UDP provides a substantially higher data transmission rate than TCP, which makes UDP indispensable for real-time network-related applications, which we study in our experiments. In our empirical study, we implement the proposed MLIN architecture on the POWDER platform [3] to investigate how network QoS degradation affects MLIN robustness. To improve MLIN robustness against network disruptions, we design a MLIN feedback mechanism and show how it can be integrated into the MLIN architecture. The proposed feedback system

Research was supported in part by the United States Military Academy (USMA) and was accomplished under Grant Number W911NF-20-1-0337 and by a grant from the U.S. Civilian Research & Development Foundation (CRDF Global) with funding from the United States Department of State. The views and conclusions contained in this document are those of the authors and should not be interpreted as representing the official policies, either expressed or implied, of the United States Military Academy (USMA), CRDF Global or the US Government. The U.S. Government is authorized to reproduce and distribute reprints for Government purposes notwithstanding any copyright notation herein. The authors wish to acknowledge the technical support provided by the University of Utah POWDER team.

gives recommendations on network adjustment actions, for example, switching to another network protocol to prevent DQ degradation during data transmission. We provide an example of how the feedback mechanism can be implemented and integrated into the proposed MLIN architecture.

The major contribution of our work is the novel MLIN feedback system that produces recommendations on actions to adjust network hyperparameters in order to improve MLIN robustness against network QoS degradation based on the ML end application performance. We propose a novel approach to consider recently emerged MLINs from the system integration perspective, and to investigate how the interrelationships between their components influence ML end application performance. Based on the experimental results, we conclude that network QoS is an important factor that directly affects ML end application performance, and adjusting network configuration may improve this performance. Instead of using only network QoS metrics to recommend network adjustment actions, we switch the focus to ML end application performance and employ it as the major indicator for MLIN network adjustment. We leverage the investigated interrelationships to concentrate the produced feedback on assuring ML end application performance to a specified user and application requirements, and on improving MLIN robustness against network disruptions.

II. Related Work

As ML classifier's robustness is critically important in multiple application areas [26], this term interpretation depends on a specific context [5], [25]. Song *et al.* [24] consider robustness as the model's ability to retain performance (test error rate) if the noise level in input data increases. Didona *et al.* define robust ML model as "less prone to error than a model based on any of the two techniques implemented alone" [8]. Carlini and Wagner [5] argue that there are two general ways to evaluate ML model's robustness: using mathematical methods (e.g., minimal required data perturbation) or initially assessing on practice (an ability of a model to provide acceptable results on noisy inputs). There are other well-established approaches addressing ML robustness. It can be considered as a user's ability to clearly interpret why a model provides one or another outcome over the specific input data sample, and relate robustness to ML interpretability. Here, studies by Lipton [14], Lou *et al.* [16], Ribeiro *et al.* [20], Rudin [22], and Ross and Doshi-Velez [21] can be mentioned. Another approach that relies on a user's trustworthiness toward the ML model as the robustness expectation is deliberated in studies by Varshney [26], Liu [15], Lakkaraju and Bastani [13], and Hutchinson *et al.* [11]. The number of studies that consider relationships between ML robustness and cyberinfrastructure components is rather limited. Hanif *et al.* [10] consider robustness in ML system design as a combination of reliability and security components.

In our work, we concentrate on MLIN robustness to network QoS degradation. Under MLIN robustness, we understand the ability of ML end applications to maintain the performance level specified by a user and application requirements in the conditions of network disruptions. We consider the network as the medium through which the data is transferred from a data source to a cloud-based ML end system. In this case, the network QoS level is directly interrelated with the quality of transmitted data, and as a consequence, it may affect the performance of data-driven ML end applications. In our solution, we leverage this interrelationship between network QoS and ML end application performance to improve MLIN robustness against network disruptions. We use ML end application performance as the major indicator, based on which the network hyperparameters should be adjusted. For example, if the ML end application performance is degraded due to a DQ drop caused by network packet losses, the solution might be to recommend switching to a more reliable network protocol, if other network QoS requirements are satisfied. Below we describe the typical MLIN architecture and the components it incorporates. Then, using a real-world example, we investigate the interrelationship between network QoS and ML end application performance we are interested in. We demonstrate how this interrelationship may be leveraged, and design an MLIN feedback system that provides recommendations on network adjustment in order to improve MLIN robustness.

III. MLIN Architecture

In this paper, we address the need to assure MLIN robustness to satisfy its specification requirements by providing the network adjustment method based on feedback from the ML end system. The feedback can be implemented in the form of recommendations to perform a certain action or a set of actions for a network operation and/or structure adjustment. The recommendations are derived based on the ML end system performance and other network QoS metrics (e.g., network packet loss). We consider the MLIN architecture that includes the following major components.

A. Data Source(s)

Data source(s) may represent the initial location where the data is generated or where physical information is first digitized. Data source(s) may or may not incorporate sensors to acquire data from the environment, as the data source may be represented as a database from which the data is derived on demand including related data processing operations. Data source(s) should incorporate a network communication interface, using which the data is transmitted over network facilities to a cloud-based ML end system. A data source may possess a local memory to store the collected data before the transmission.

B. Network Facilities

Network facilities are shared by all nodes, forming a part of a transmission system comprised of network stations and the transmission lines connecting them. The network facilities are used to transmit the data from data source(s) to a cloud-based ML end system. In addition, they should provide QoS

evaluation per specified time intervals. QoS can include any network characteristics which are relevant based on the user or application requirements. The network facilities also should allow reconfiguration by changing the specified hyperparameters (e.g., transport protocol, communication channel bandwidth, etc.) according to user and application requirements.

C. Cloud-Based ML End System

Cloud-based ML end systems are used to perform classification, detection, or recognition patterns in the data originating from data source(s). A ML end system may be installed on a remote server accessible over a network, which may be a part of a cloud and network infrastructure. It may leverage any ML model (e.g., artificial neural networks, convolution neural networks, etc.) used for the classification, detection, or recognition of patterns in the received data. The employed ML model may be pre-trained and further re-trained on data of any type, format, and structure, which are determined by the user and application requirements.

D. Performance Evaluation Component

This component is responsible for evaluating cloud-based ML end system operational performance against the performance specification. Performance evaluation could be done internally by the ML end system itself or may be realized as separate software, hardware, a combination thereof, and other infrastructure employed to maintain its operation. Performance evaluation could be performed based on the set of metrics (e.g., accuracy, classification error, true positive rate, etc.). The metrics choice is determined by the user and application requirements, and by the employed ML end system. The ML end performance is evaluated against the ML end application performance specification. This specification establishes the performance level to which the ML end system should correspond. For example, if the accuracy metric is used to evaluate the ML end system's decision-making performance, the specification may set up the acceptable level of no less than 95% for this metric.

E. Proposed Feedback for MLIN Robustness Assurance

The developed integrated structural MLIN organization defines the MLIN system's structure, how the components interact with each other on the system level, and how these interactions influence the ML end application performance. The proposed feedback recommendation system produces recommendations on actions to change network hyperparameters. The system may consist of software, hardware, a combination thereof, and any other infrastructure required to maintain its operation. The feedback system accepts the ML system performance demonstrated over the processed data as an input, provided by the performance evaluation component. Another input is composed of the network QoS metrics' values, evaluated over the data transmission from the data source(s) to the cloud-based ML end system. Examples of such QoS metrics may be network packet loss rate, communication channel bandwidth measurements, etc. The feedback system's output may be produced in a form of a recommendation on a certain action or a set of adjustment actions. The produced recommendations may depend on a particular network structure (e.g., network topology, configuration, data transmission technology, etc.) and are determined by the user and application requirements. Examples of the recommendations on actions may be switching to another network transport protocol or increasing communication channel bandwidth.

(a) (b) (c)

Fig. 1. Examples of images transmitted over a network with: (a) – 128B buffer and 1% packet loss; (b) – 256B buffer size and 5% packet loss; (c) – 512B buffer and 20% packet loss

IV. Empirical Study

In this section, we demonstrate an example of how the proposed MLIN architecture may be implemented in a real-world use-case scenario. We investigate the interrelationship between network characteristics and ML end application performance. In particular, we study how network packet loss affects the robustness of open-source industrial ML image classifiers against DQ degradation due to network disruptions. Based on the obtained interrelationships, we derive rules for network hyperparameters adjustment. As an example of the MLIN feedback recommendation system implementation, we design a rule-based recommendation system, that provides network hyperparameters adjustment actions in order to improve MLIN robustness. To facilitate our experiments, we employ the POWDER testbed [4], which is a wireless and mobile network research platform deployed in a real-world city-scale environment, to obtain images corrupted in the process of their transmission between real network nodes.

To study how network disruptions affect ML image classification, we employ a subset of traffic sign images taken from the Open Images V6 dataset [1]. The dataset contains images labeled for classification, object detection, and semantic segmentation. The first step included transferring a set of images using UDP between two network nodes. Since UDP is an unreliable protocol, the transferred images appeared to be corrupted on the receiving endpoint. To recreate network service degradation conditions, two parameters are investigated during the image transmission: the buffer size and the packet loss.

The buffer size of a UDP socket determines the amount of data that could be stored and transferred in one go from the sender side. While the performance of an object detector strongly depends on the object location in either cropped or missed regions, the image corruption caused by packet loss represents a more interesting case investigated in this study.

The packet loss in a network represents the ratio of packets that are dropped during communication, and can be calculated according to (1).

$$PLR = \frac{N^{tx} - N^{rx}}{N^{tx}} \times 100\%, \quad (1)$$

where N^{tx} and N^{rx} are the total number of transmitted and received packets [19]. Since in the POWDER platform data are transmitted over the real terrain, there is some inherent and underlying packet loss in each transmission. To obtain the corrupted images, we manually change the packet loss percentage while media files transmission in the range between 1 and 20%. Examples of the transmitted images with various buffer sizes and packet loss rates are shown in Figure 1.

We employ image classification models such as VGG16, InceptionV3, and EfficientNet as ML end applications. These models are state-of-the-art ML image classifiers and have proven their robustness as they are trained over ImageNet dataset [7]. ImageNet contains around 15 million images spread among 1000 classes. Any models that are trained and perform well on ImageNet dataset are bound to be robust on original images.

From [1] we employ two image categories, which are originally unbalanced as the number of images for the traffic sign class is larger than the stop sign class. In order to train the image classification model efficiently, the image set needs to be balanced among both classes. Therefore, we selected 450 images of both classes randomly. Then we developed an annotation system, which crops out the specific portion of the original image consisting of either stop or traffic signs, and saves these extracted sections to their target location. However, 450 images might not be enough to achieve acceptable performance in the case of training a classification model from scratch. To overcome this issue, we apply transfer learning.

Originally, the parameters for our image classification models are imported from the comprehensive ImageNet dataset [7]. In order to make these models work as binary classifiers between stop and traffic signs, we remove their final layer and employ the transfer learning technique. We perform training over our image subsets only at the final layer. This way the efforts and training time of the model is negligible compared to if we had to train the initial model. We trained the models and tested their accuracy. The models achieved the following train accuracy values: 92.24% for VGG16, 89.27% for InceptionV3, and 87.92% for EfficientNet. After the transfer learning stage, the models demonstrated the following transfer accuracy results: 93.24% for VGG16, 93.92% for InceptionV3, and 95.95% for EfficientNet. Train accuracy here evaluates the classifier performance on images submitted to the classifier models, which have been pre-trained on the generic training set. Transfer accuracy evaluates the classifier's performance after it has been further trained on images similar to the submitted ones. After the transfer learning stage, we re-evaluated the ML models' performance on the test image set obtained from the POWDER testbed. Below we discuss the performance of ML image classifiers achieved over images transmitted in various network QoS conditions.

TABLE I
RESULTS ON ML PERFORMANCE FOR REAL IMAGES CLASSIFICATION CASE

Image type	Retention (%)	Models	Acc† (%)	Rel. Acc‡ (%)	Avg. file size (kB)
Original	100	VGG16	93.24	–	135
		Inception	93.92	–	
		EfficientNet	95.95	–	
128B*, PL** 1%	95	VGG16	80.9	76.85	–
		Inception	85.07	80.82	
		EfficientNet	84.42	79.25	
128B, PL 2%	85	VGG16	76.26	64.82	
		Inception	78.06	76.35	
		EfficientNet	79.5	67.57	
128B, PL 5%	75	VGG16	64.9	48.67	
		Inception	67.76	50.82	
		EfficientNet	64.49	48.36	
256B, PL 5%	80	VGG16	72.57	58.05	
		Inception	75.95	60.76	
		EfficientNet	74.26	59.4	
256B, PL 10%	75	VGG16	59.2	44.4	
		Inception	63.18	47.38	
		EfficientNet	62.69	47.01	
512B, PL 1%	95	VGG16	89.9	85.4	134
		Inception	88.85	84.4	
		EfficientNet	90.94	86.39	
512B, PL 5%	85	VGG16	78.65	74.71	130
		Inception	82.77	78.63	
		EfficientNet	77.53	73.65	
512B, PL 10%	80	VGG16	65.02	52.01	122
		Inception	66.67	53.33	
		EfficientNet	67.49	53.99	
512B, PL 20%	75	VGG16	62.16	46.62	100
		Inception	61.08	45.81	
		EfficientNet	65.95	49.46	

* Buffer size; ** Packet loss percentage; † Accuracy; ‡ Relative Accuracy

V. EXPERIMENTAL RESULTS AND EVALUATION

Table I represents accuracy values for numerous buffer size and packet loss combination samples. For smaller buffer sizes, increasing packet loss results in image file corruption, and image classification models are not able to assign a label to these images. The "Retention" column in Table I lists the ratio of images that the model was able to classify. The "Relative Accuracy" column takes into account the retention ratio in recalculating the classifier's accuracy, so it shows a further accuracy drop. Data are entered for retention value less than 100%.

As it can be seen in Table I, in the case of buffer scarcity (128B), even 1% packet loss may result in more than 10% drop in the accuracy of ML classifiers such as VGG16 and EfficientNet. In the meantime, with a larger buffer size (512B) the accuracy of ML classifiers decreases by at most 5%. For the 5% packet loss, we can see that the accuracy is still better than 75% for all the models. However, 20% packet loss decreases the accuracy of classifiers by more than 30%. From the column "Avg. file size" in Table I, one can also see that packet loss results in a significant drop in the image file size as well.

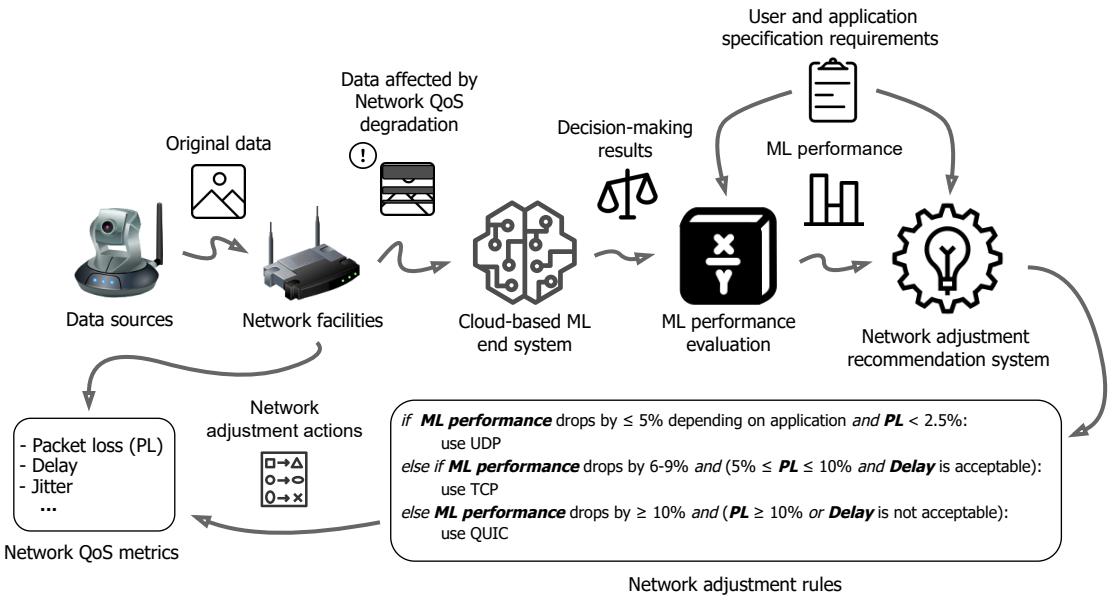

Fig. 2. An example of the proposed feedback recommendation system for MLIN robustness assurance

Our results show that network QoS degradation may result in a significant drop in the accuracy of ML classifiers in the context of MLINs, and may lead to potential misclassifications. Misclassifications are particularly unacceptable for intelligent systems that are employed in transportation, healthcare, or security infrastructure.

VI. MLIN Feedback Recommendation System

In the previous section, we showed how the network QoS degradation led to ML image classification performance degradation. We observe that packet loss during the network transmission results in a MLIN robustness drop, and this drop can be critical even in the case of small packet loss rates. In this section, we propose using the ML end application performance to provide feedback on MLIN network adjustment in order to assure MLIN robustness. In particular, we provide an example of how the recommendations on network parameters adjustment can improve the quality of transmitted data, and, as a consequence, assure MLIN robustness to specified user and application requirements.

Since many modern smart technologies rely on remote ML end applications, it is important to improve network robustness during packet loss spikes to mitigate the consequences of network congestion or malicious attacks such as DDoS. The outcome of packet losses depends on the transport protocol used for network communication. Although UDP protocol is actively used for data transmission in real-time applications, it is unreliable by its design and subject to a loss of network packets. Packet loss leads to irreversible data losses, and, as we demonstrate in our empirical study, affects the performance of MLIN. ML end systems' robustness can be addressed from various perspectives, such as transfer learning, data cleaning, wrangling, etc. However, these approaches are concentrated on a single component and do not consider ML end system as the integration of its components. In contrast, our approach is based on investigating the interrelations between these components. In our experiments, we demonstrate that network conditions (e.g., packet loss percentage) directly impact the performance of ML end classifiers. To address the MLIN robustness problem, we propose a MLIN feedback recommendation system for network hyperparameters adjustment.

One solution to the problem of DQ degradation prevention during network disruptions might be switching network transport protocols. For example, in contrast to UDP, TCP protocol preserves the quality of data over transmission. However, it significantly increases data transfer time, which can be intolerable in real-time applications. Another option is an application of the QUIC network protocol that may outperform TCP in terms of data transfer time when the packet loss ratio exceeds 10% [29]. At the same time, QUIC can support a reliable media streaming service, unlike the UDP protocol which could provide faster service. QUIC protocol would be our recommendation for the transport protocol under poor network conditions. However, additional research is required to provide robust recommendations, as they would depend on a number of factors related to a specific ML system, data type, and user and application requirements.

However, we not only recommend shifting the communication protocol to QUIC but also switching between QUIC and UDP in case of network QoS changes. The adjustment actions have to be determined by the ML application specification and user requirements. An example of the MLIN feedback mechanism realization is presented in Figure 2. The proposed structure implements the operational control of the network infrastructure. It includes the set of rules that needed to be set up by a user based on the application domain and specification

requirements.

VII. Conclusion

We propose a novel approach to improve the robustness of recently emerged MLINs against possible network QoS and DQ degradation in real applications. The major contribution of our work is the MLIN feedback system that provides recommendations on network adjustment actions based on ML end application performance in order to improve MLIN robustness to network QoS degradation. In contrast to other approaches, we consider and describe MLIN from the system integration perspective, which allows us to establish the interrelationships between MLIN components and employ them for MLIN network restructuring. We operate with high-level definitions and terms, which makes our approach applicable to various ML end application domains regardless of the employed ML models and data type. We rely on ML end application performance as the major indicator for requiring MLIN network adjustments to improve MLIN robustness against DQ degradation. This allows system components restructuring with a focus on the imperative ML end performance indicator. Our experiments confirm that the network QoS has a high impact on the ML end application performance. We design a MLIN feedback recommendation system for network hyperparameters adjustment, and we demonstrate its realization in real-world experiments. The design we propose is flexible and allows users to implement their own techniques to develop recommendations on MLIN network adjustment.

References

[1] Overview of Open Images V6. (Date last accessed 28-July-2021). [Online]. Available: https://storage.googleapis.com/openimages/web/factsfigures.html

[2] "Andrew Ng Launches a Campaign For Data-Centric AI," 2021, accessed on April 04, 2022. [Online]. Available: https://www.forbes.com/sites/gilpress/2021/06/16/andrew-ng-launches-a-campaign-for-data-centric-ai/?sh=23e0624c74f5

[3] J. Breen, A. Buffmire, J. Duerig, K. Dutt, E. Eide, A. Ghosh, M. Hibler, D. Johnson, S. K. Kasera, E. Lewis *et al.*, "Powder: Platform for open wireless data-driven experimental research," *Computer Networks*, p. 108281, 2021.

[4] J. Breen, J. Duerig, E. Eide, M. Hibler, D. Johnson, S. Kasera, D. Maas, A. Orange, N. Patwari, R. Ricci *et al.*, "Mobile and wireless research on the powder platform," in *Proceedings of the 19th Annual International Conference on Mobile Systems, Applications, and Services*, 2021, pp. 509–510.

[5] N. Carlini and D. Wagner, "Towards evaluating the robustness of neural networks," in *2017 ieee symposium on security and privacy (sp)*. IEEE, 2017, pp. 39–57.

[6] S. Chuprov, L. Reznik, A. Obeid, and S. Shetty, "How degrading network conditions influence machine learning end systems performance? (*will be published soon*)," in *The 9th International Workshop on Computer and Networking Experimental Research using Testbeds (CNERT)*. IEEE, 2022, pp. 1–6.

[7] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "Imagenet: A large-scale hierarchical image database," in *2009 IEEE conference on computer vision and pattern recognition*. Ieee, 2009, pp. 248–255.

[8] D. Didona, F. Quaglia, P. Romano, and E. Torre, "Enhancing performance prediction robustness by combining analytical modeling and machine learning," in *Proceedings of the 6th ACM/SPEC international conference on performance engineering*, 2015, pp. 145–156.

[9] M. Fan, Y. Liu, C. Chen, S. Yu, W. Guo, L. Wang, and X. Liu, "Towards evaluating the reliability of deep neural networks based iot devices," *IEEE Internet of Things Journal*, 2021.

[10] M. A. Hanif, F. Khalid, R. V. W. Putra, S. Rehman, and M. Shafique, "Robust machine learning systems: Reliability and security for deep neural networks," in *2018 IEEE 24th International Symposium on On-Line Testing And Robust System Design (IOLTS)*. IEEE, 2018, pp. 257–260.

[11] B. Hutchinson, A. Smart, A. Hanna, E. Denton, C. Greer, O. Kjartansson, P. Barnes, and M. Mitchell, "Towards accountability for machine learning datasets: Practices from software engineering and infrastructure," in *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency*, 2021, pp. 560–575.

[12] A. A. Laghari, R. A. Laghari, A. A. Wagan, and A. I. Umrani, "Effect of packet loss and reorder on quality of audio streaming," *EAI Endorsed Transactions on Scalable Information Systems*, vol. 7, no. 25, 2020.

[13] H. Lakkaraju, E. Kamar, R. Caruana, and J. Leskovec, "Faithful and customizable explanations of black box models," in *Proceedings of the 2019 AAAI/ACM Conference on AI, Ethics, and Society*, 2019, pp. 131–138.

[14] Z. C. Lipton, "The mythos of model interpretability: In machine learning, the concept of interpretability is both important and slippery." *Queue*, vol. 16, no. 3, pp. 31–57, 2018.

[15] H. Liu, "Trustworthy machine learning: Fairness and robustness," in *Proceedings of the Fifteenth ACM International Conference on Web Search and Data Mining*, 2022, pp. 1553–1554.

[16] Y. Lou, R. Caruana, and J. Gehrke, "Intelligible models for classification and regression," in *Proceedings of the 18th ACM SIGKDD international conference on Knowledge discovery and data mining*, 2012, pp. 150–158.

[17] J. P. McManus, T. G. Day, and Z. J. Mailloux, "The effects of latency, bandwidth, and packet loss on cloud-based gaming services," *Interact Qualif Proj (All Years)*, vol. 58, 2019.

[18] Š. Mrvelj and M. Matulin, "Impact of packet loss on the perceived quality of udp-based multimedia streaming: a study of user quality of experience in real-life environments," *Multimedia systems*, vol. 24, no. 1, pp. 33–53, 2018.

[19] M. S. Obaidat, F. Zarai, and P. Nicopolitidis, *Modeling and simulation of computer networks and systems: Methodologies and applications*. Morgan Kaufmann, 2015.

[20] M. T. Ribeiro, S. Singh, and C. Guestrin, "" why should i trust you?" explaining the predictions of any classifier," in *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining*, 2016, pp. 1135–1144.

[21] A. Ross and F. Doshi-Velez, "Improving the adversarial robustness and interpretability of deep neural networks by regularizing their input gradients," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, 2018.

[22] C. Rudin, "Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead," *Nature Machine Intelligence*, vol. 1, no. 5, pp. 206–215, 2019.

[23] M. Salah ud din, M. A. Ur Rehman, R. Ullah, C.-W. Park, D. H. Kim, and B. seo Kim, "Improving resource-constrained iot device lifetimes by mitigating redundant transmissions across heterogeneous wireless multimedia of things," *Digital Communications and Networks*, 2021. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S235286482100064X

[24] H. Song, M. Kim, and J.-G. Lee, "Selfie: Refurbishing unclean samples for robust deep learning," in *International Conference on Machine Learning*. PMLR, 2019, pp. 5907–5915.

[25] H. Song, M. Kim, D. Park, Y. Shin, and J.-G. Lee, "Robust learning by self-transition for handling noisy labels," in *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*, 2021, pp. 1490–1500.

[26] K. R. Varshney, "Trustworthy machine learning and artificial intelligence," *XRDS: Crossroads, The ACM Magazine for Students*, vol. 25, no. 3, pp. 26–29, 2019.

[27] Y. Wang, V. Menkovski, I. W.-H. Ho, and M. Pechenizkiy, "Vanet meets deep learning: The effect of packet loss on the object detection performance," in *2019 IEEE 89th Vehicular Technology Conference (VTC2019-Spring)*. IEEE, 2019, pp. 1–5.

[28] H. Xu, Y. Ma, H.-C. Liu, D. Deb, H. Liu, J.-L. Tang, and A. K. Jain, "Adversarial attacks and defenses in images, graphs and text: A review," *International Journal of Automation and Computing*, vol. 17, no. 2, pp. 151–178, 2020.

[29] Y. Yu, M. Xu, and Y. Yang, "When quic meets tcp: An experimental study," in *2017 IEEE 36th International Performance Computing and Communications Conference (IPCCC)*. IEEE, 2017, pp. 1–8.